

A Simple and Efficient Algorithm for Finding Minimum Spanning Tree Replacement Edges

David A. Bader, Paul Burkhardt

<http://jgaa.info/> vol. 26, no. 4, pp. 577-588 (2022)

Nagy Bence

2025. december 1.

Áttekintés: minimális feszítőfa, csereél

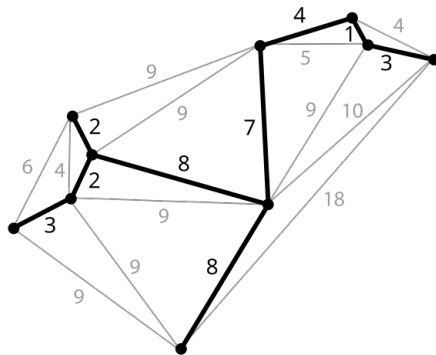
Definíció:

Legyen $G = (V, E)$ egy irányítatlan, súlyozott gráf, ahol $n = |V|$ és $m = |E|$, és minden $e \in E$ élhez tartozik egy $w(e)$ súly. Egy minimális feszítőfa $T = MST(G)$ a gráf olyan $n-1$ élből álló részhalmaza, amely összeköti az összes csúcsot, és az élek összsúlya minimális.

Definíció:

A csereél az a legkisebb súlyú él, amely újra összeköti a minimális feszítőfát egy minimális feszítőfabeli él törlése után.

Probléma alkalmazása: utakon balesetek, kommunikációs hálózatokban kábel szakadás



Kép forrás: Wikipédia

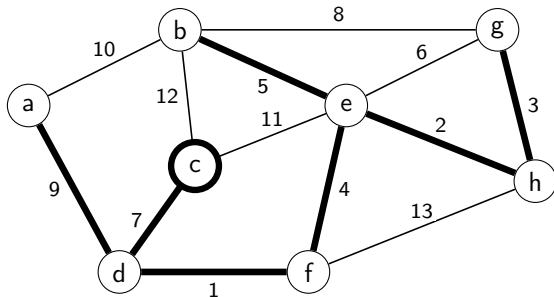
- Adottak:
 - A gráf minimális feszítőfája
 - A nem feszítőfabeli élek RENDEZVE
- Keressük:
 - Csereéleket minden minimális feszítőfabeli élhez.

- Az algoritmus képes megtalálni minden csereélet, ha a nem min feszítőfabeli élek élsúly szerint növekvő sorrendben rendezve megvannak határozva:
 - **Idő- és tárbonyolultság:** $\mathcal{O}(m+n)$
(m : élek száma, n : csúcsok száma)
- A cikk előtt lehetett tudni, hogy ez elméletileg lehetséges, de ez az első algoritmikus megoldás erre lineáris időkomplexitással.

- 1975: Spira és Pan: MST egy éléhez csereél $\mathcal{O}(n^2)$ időben.
- 1978: Chin és Houck: MST minden éléhez csereél $\mathcal{O}(n^2)$ időben.
- 1979: Tarjan: $\mathcal{O}(m\alpha(m, n))$
 - α : Ackermann függvény inverze: nagyon-nagyon lassan nő a bemenethez képest.
 - pl: $\alpha(7) = 2$, $\alpha(61) = 3$, $\alpha(2^{2^{2^{2^{2^2}}}} - 3) = 4$
 - Majdnem lineáris megoldás, de mégsem az.
- 2022: Bader és Burkhardt: $\mathcal{O}(m + n)$

Az algoritmus alapötlete 1.

- Adott T és a nem-fa élek $E \setminus E_T$ súly szerint növekvő sorban.
- Figyeljük meg, hogy mindegyik $m - n + 1$ darab él $E \setminus E_T$ -ben egy fundamentális kört indukál T élével.
- Csereél a fa egy konkrét éléhez: az a legkisebb súlyú él, amely egy olyan kört indukál a fán, amibe benne van az adott MST él.



Itt: az (e,g) él az (e,h) és a (g,h) élek minimális költségű csereéle.

Az algoritmus alapötlete 2.

- 1 Kijelölünk egy tetszőleges csúcsot a fa gyökerének.
- 2 A mélységi bejárás alapján minden csúcsnak rendeljük hozzá a szülőjét, valamint a belépési ($IN[v]$) és kilépési ($OUT[v]$) sorszámát.
- 3 Járjuk be a fundamentális kört, amelyet egy nem-fa él indukált.
- 4 Hajtsunk végre úttömörítés (path compression), hogy kihagyjuk azon élek bejárását, amelyenek már találtunk csereélet.

A diszjunkt halmaz adatszerkezetet használjuk gyors úttömörítésre a Gabow-Tarjan-féle statikus uniófa módszer alapján.

Definíció:

A diszjunkt halmaz adatstruktúra diszjunkt halmazok gyűjteményét tartja fenn:

$\mathcal{S} = \{S_1, S_2, \dots, S_k\}$. Minden halmazt egy **képviselőjével** azonosítjuk, amely tagja a halmaznak.

Műveleteket a diszjunkt halmaz adatstruktúrán:

- **makeset(v)**: létrehoz egy új egy elemű halmazt: $\{v\}$, amelynek v lesz a képviselője.
- **find(v)**: visszaadja annak a diszjunkt halmaznak a képviselőjét, amelyben v van.
- **link(u, v)**: Létrehoz egy új halmazt, ami u és v halmazának az uniója.

Gabow és Tarjan algoritmus a diszjunkt halmazok egy speciális esetére

- Speciális eset: a teljes uniófa (Union Tree) ismert az algoritmus indulásakor.
- Esetünkben az eredeti MST lesz a Union-Tree.
- Az algoritmus **m darab unió és find műveletet** hajt végre, **n darab elemen**: $\mathcal{O}(m+n)$ futási időben, $\mathcal{O}(n)$ tárhelykomplexitással
- Itt: $\text{link}(v)$: v halmazát és $P[v]$ halmazát összeuniózza és $P[v]$ képviselője lesz az új halmaz képviselője. ($P[v]$: v szülője a Union-Tree-ben)

Az algoritmus pszeudokódja (inicializációk és pathlabel meghívása)

- 1: Root the MST T at arbitrary vertex v_r and store parents in P .
- 2: $P[v_r] \leftarrow v_r$ ▷ root's parent points to root
- 3: Run DFS on T , setting $IN[v]$ and $OUT[v]$ to the counter value when v is first and last visited.
- 4: **for all** vertices $v \in V$ **do**
- 5: $\text{MAKESET}(v)$ ▷ Initialize the disjoint sets
- 6: **for all** edges $e \in E_T$ **do**
- 7: $R_e \leftarrow \emptyset$ ▷ Initialize the replacement edges
- 8: **for** $k \leftarrow 1$ **to** $m - n + 1$ **do**
- 9: $(v_i, v_j) \leftarrow e_k$ ▷ Scan the $m - n + 1$ sorted non-MST edges
- 10: $\text{PATHLABEL}(v_i, v_j, (v_i, v_j))$
- 11: $\text{PATHLABEL}(v_j, v_i, (v_i, v_j))$

Az algoritmus pszeudokódja (PathLabel metódus)

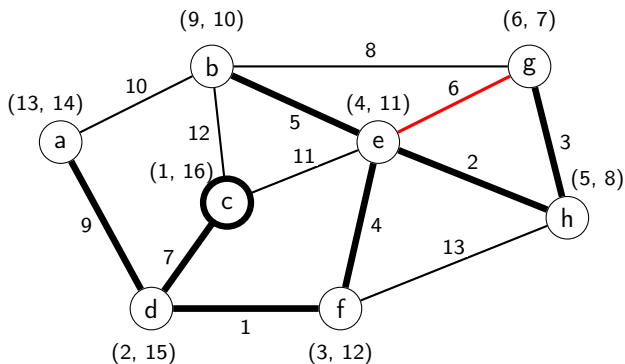
```
1: procedure PATHLABEL( $s, t, e$ )
2:   if  $IN[s] < IN[t] < OUT[s]$  then                                ▷  $s$  is ancestor of  $t$ 
3:     return
4:   if  $IN[t] < IN[s] < OUT[t]$  then                                ▷  $t$  is ancestor of  $s$ 
5:      $PLAN \leftarrow ANCESTOR$ ;  $k_1 \leftarrow IN[t]$ ;  $k_2 \leftarrow IN[s]$ 
6:   else
7:     if  $IN[s] < IN[t]$  then
8:        $PLAN \leftarrow LEFT$ ;  $k_1 \leftarrow OUT[s]$ ;  $k_2 \leftarrow IN[t]$       ▷  $s$  is left of  $t$ 
9:     else
10:       $PLAN \leftarrow RIGHT$ ;  $k_1 \leftarrow OUT[t]$ ;  $k_2 \leftarrow IN[s]$     ▷  $s$  is right of  $t$ 
11:    $v \leftarrow s$ 
12:   while  $k_1 < k_2$  do                                           ▷ Detecting when below LCA( $s, t$ )
13:     if  $FIND(v) = v$  then                                         ▷ If true, set replacement edge for ( $v, P[v]$ )
14:        $R(v, P[v]) \leftarrow e$                                      ▷ Set the replacement edge
15:        $LINK(v)$                                                   ▷ Union the disjoint sets of  $v$  and  $P[v]$ 
16:        $v \leftarrow FIND(v)$ 
17:   switch  $PLAN$  do
18:     case  $ANCESTOR$ :
19:        $k_2 \leftarrow IN[v]$ 
20:     case  $LEFT$ :
21:        $k_1 \leftarrow OUT[v]$ 
22:     case  $RIGHT$ :
23:        $k_2 \leftarrow IN[v]$ 
```

- Meghatározza s és t élek pozícióját egymáshoz képest az MST-n. (LEFT, RIGHT, ANCESTOR)
- s éllel kezdve végig megyünk az $s \rightarrow t$ úton, és:
 - Ha az adott él még nem volt path compression-elve ($v = \text{find}(v)$), akkor annak az élnek az inputként megadott élet menti el csereélének. Majd path compression-t hajt végre ($\text{link}(v)$).
 - k_1 és k_2 counter-eket léptetgeti, ameddig végig nem érünk az $s \rightarrow t$ úton.

Példa lefutás - első nem-fa él (e, g)

Első nem-fa él: (e, g)

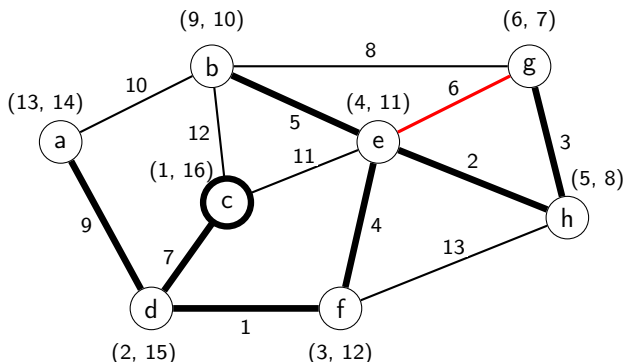
- Az e él g élnek a felmenője a fában, így megkapjuk $plan = ANCESTOR$ -t. Továbbá $k_1 = IN[e] = 4$ és $k_2 = IN[g] = 6$, és mivel $k_1 < k_2$ ezért elkezdjük a bejárást.
- A kör bejárás g -nél kezdjük. Mivel g -t még nem jártuk be ($link(g) = g$), ezért a (g, e) nem-MST élet (g, h) csereéleként mentjük el ($R(g, h) = (e, g)$).
- g és h diszjunkt halmazait link-eljük úgy, hogy g diszjunkt halmaza kapja meg h képviselőjét. Ez a lépés tömöríti a (g, h) részt.



Példa lefutás - első nem-fa él (e, g) folyt.

Első nem-fa él (e, g) : (folyt.)

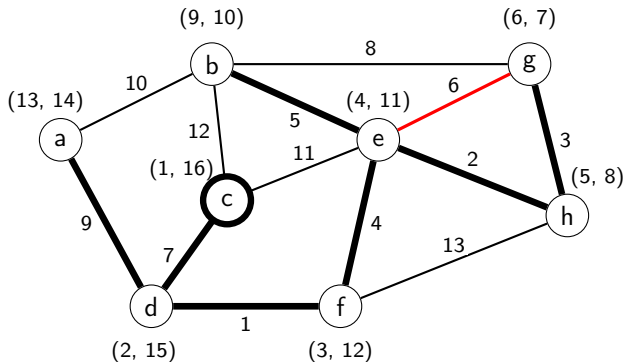
- A következő él a bejárásban h lesz, mivel ő g szülője ($\text{find}(g) = h$). k_2 -t frissítjük $\text{IN}[h]$ -vel. ($\text{IN}[h] = 5$)
- Mivel (e, h) él még nem lett tömörítve ($\text{find}(h) = h$), ezért (h, e) csereélenek (g, e) élt adjuk.
- h és e diszjunkt halmazait link-eljük úgy, hogy h diszjunkt halmaza kapja meg e képviselőjét. Ez tömöríti a (g, h, e) részutat.
- A bejárásban e lesz a következő él, szóval k_2 megkapja $\text{IN}[e]$ -t ($= 4$), már egyenlő lesz k_1 -el ($= 4$), ezzel leállítva a while ciklust.



Példa lefutás - első nem-fa él (e, g) folyt.

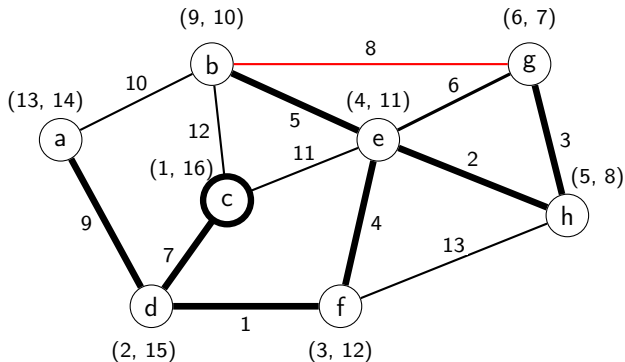
Első nem-fa él (e, g) : (folyt.)

- A fordított (e, g) éllel PathLabel hívás nem fog feldolgozódni, mivel e felmenője g -nek.



Második nem-fa él ($b-g$):

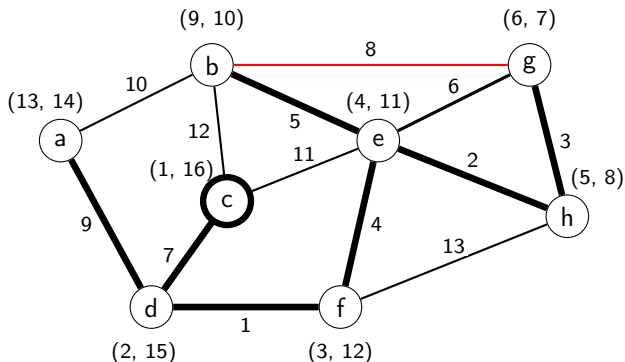
- Meghívjuk $\text{PathLabel}(b, g, (b, g))$ -t
- RIGHT plan-t kapjuk meg. $k_1 = \text{OUT}[g]$, $k_2 = \text{IN}[b]$, szóval $k_1 < k_2$.
- A bejárást b -vel kezdjük és mivel b -t még nem látogattuk meg ezért (b, e) él a (b, g) nem-MST élt kapja meg csereélként.
- b és e diszjunkt halmazait link-eljük. Ezzel tömörítjük a (b, e) részutat.



Példa lefutás - második nem-fa él ($b-g$)

Második nem-fa él ($b-g$): (folyt.)

- A következő él a bejárásban e (mivel ő b-nek a szülője), így $k_2 = \text{IN}[e]$. Ekkor $k_2 < k_1$ teljesül, így leáll a while loop.
- Meghívjuk PathLabel-t ($g-b$) éllel: $\text{PathLabel}(g, b, (g, b))$.
- Megkapjuk LEFT ágtervet, $k_1 = \text{OUT}[g]$ és $k_2 = \text{IN}[b]$ -vel, szóval $k_1 < k_2$ és megkezdjük a bejárást g-vel kezdve.
- Ekkor figyeljük meg, hogy a diszjunkt halmazok tömörítették már a (g, h, e) részutat, ezért $\text{find}(g) \neq g$



Tétel:

Adott egy irányítatlan, súlyozott $G = (V, E)$ gráf minimális feszítőfája, és a nem feszítőfabeli élek súly szerint rendezve, akkor az algoritmus $\mathcal{O}(m + n)$ időben és $\mathcal{O}(m + n)$ helyigény mellett megkeresni G minimális feszítőfájának összes minimális költségű csereélét.

Bizonyítás:

Legyen T a G gráf minimális feszítőfája.

- 1 A P szülő tömb inicializálása: $\mathcal{O}(n)$
- 2 Mivel T -ben $n - 1$ él van, ezért a DFS lefuttatása T -n az IN és OUT értékek inicializálásához: $\mathcal{O}(n)$.
- 3 Csereélék inicializálása minden MST élhez: $\mathcal{O}(n)$ idő.
- 4 $m - n + 1 = \mathcal{O}(m)$ nem-MST él olvasás van, növekvő sorban súly szerint: $\mathcal{O}(m)$ időkomplexitás.

Bizonyítás: (folyt.):

- **Definíció:** Az az él, amelynek eltávolítása a legnagyobb növekedést okozza az MST (minimális feszítőfa) súlyában.
- **Az algoritmus hozzájárulása:**
 - A csereélek meghatározása után a legfontosabb él kiválasztható $\mathcal{O}(n)$ időben. Csak meg kell keresni, hogy melyik MST-beli él súlyának a különbsége legnagyobb a csereélének súlyával.
 - Így: a legfontosabb él kiszámítható $\mathcal{O}(m + n)$ időben, ha adott az eredeti MST és a nem MST élek rendezve vannak.

- Képesek vagyunk egy minimális feszítőfa minden élének a csereélét megtalálni $\mathcal{O}(m + n)$ időben, ha a nem MST éleket növekvő sorrendben kapjuk meg.
- Ehhez Gabow és Tarjan speciális diszjunkt halmaz adatszerkezetét használjuk fel.

Köszönöm a figyelmet!